

Задача.

Я как Куратор курса хочу иметь возможность просмотреть отчет по посещению студентами занятий и их успеваемости чтобы принимать решение о поощрении

1. User Story

Как Куратор курса, я хочу иметь возможность просмотреть отчет по посещаемости студентов и их успеваемости, чтобы принимать решения о поощрении.

Критерии приемки:

1. Куратор может войти в систему с использованием своих учетных данных.
2. В системе доступен раздел с отчетами по курсу.
3. Отчет содержит данные о посещаемости каждого студента за выбранный период времени.
4. Отчет содержит данные об успеваемости каждого студента (оценки, выполненные задания).
5. Куратор может скачать отчет в формате PDF или Excel для дальнейшего использования.

2. Use Case

Название: Просмотр отчета по посещаемости и успеваемости студентов.

Актеры: Куратор курса (основной), Система (второстепенный).

Основной сценарий:

1. Куратор входит в систему, используя свои учетные данные.
2. Переходит в раздел "Отчеты".
3. Выбирает курс, за который хочет получить отчет.
4. Указывает период времени (например, последние 3 месяца).
5. Нажимает кнопку "Сформировать отчет".
6. Система отображает данные о посещаемости и успеваемости студентов в виде таблицы.
7. Куратор может просмотреть отчет на экране или скачать его в формате PDF/Excel.

Альтернативный сценарий:

Если данные за указанный период отсутствуют, система выводит сообщение: "Данные за выбранный период отсутствуют".

3. Требования

Бизнес-требования:

1. Система должна предоставлять кураторам курсов доступ к данным о посещаемости и успеваемости студентов для принятия решений о поощрении.
2. Отчеты должны быть точными, актуальными и легко интерпретируемыми.
3. Доступ к данным должен быть ограничен ролями, чтобы только кураторы могли видеть соответствующую информацию.

Бизнес-правила:

1. Данные о посещаемости формируются на основе отметок преподавателей о присутствии студентов на занятиях.
2. Успеваемость студентов рассчитывается на основе их оценок за выполненные задания и тесты.
3. Куратор может видеть только данные студентов своего курса.

Пользовательские требования:

1. Пользователь (Куратор) должен иметь возможность формировать отчет за любой указанный период времени.

Функциональные требования:

1. Система должна поддерживать авторизацию пользователей с разными ролями (Куратор, Преподаватель, Студент).
2. Система должна предоставлять возможность выбора периода времени для формирования отчета.
3. Система должна отображать данные о посещаемости студентов в виде таблицы с указанием дат и статусов (Присутствовал/Отсутствовал).
4. Система должна отображать данные об успеваемости студентов (оценки за задания, тесты, итоговый балл).
5. Система должна позволять скачивать отчет в формате PDF или Excel.

Нефункциональные требования:

1. Время генерации отчета не должно превышать 5 секунд при запросе данных за 6 месяцев.
2. Система должна быть доступна 99% времени в течение учебного года.
3. Интерфейс должен быть интуитивно понятным и адаптированным для работы как на компьютере, так и на мобильных устройствах.

Требования к системе:

- Доступность актуальных данных о посещаемости и успеваемости.
- Удобный интерфейс для просмотра и фильтрации отчетов.
- Возможность сохранения и печати отчетов.

- Проверка прав доступа к куратору для ограничения доступа к курсам.

4. Моделирование

Вариантов использования (Use case)

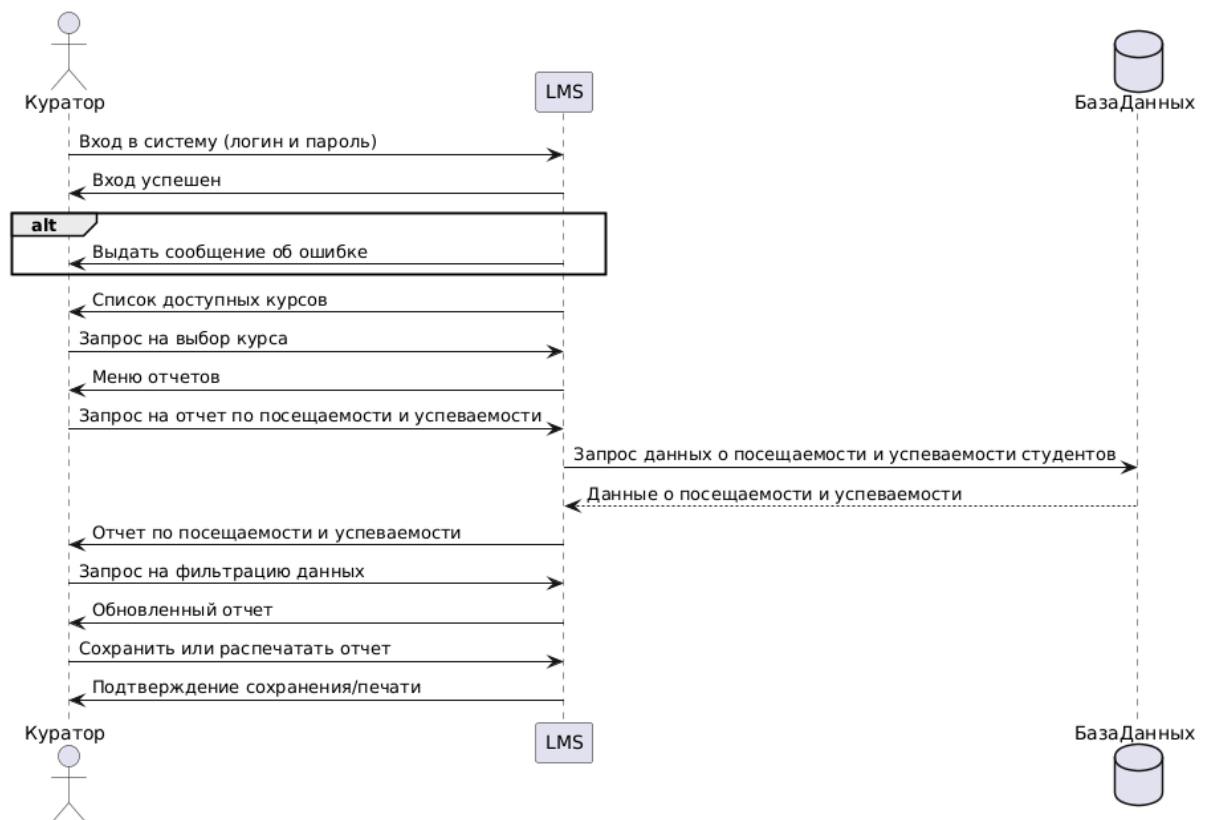
Последовательности (Sequence)

Активности (Activity) или BPMN

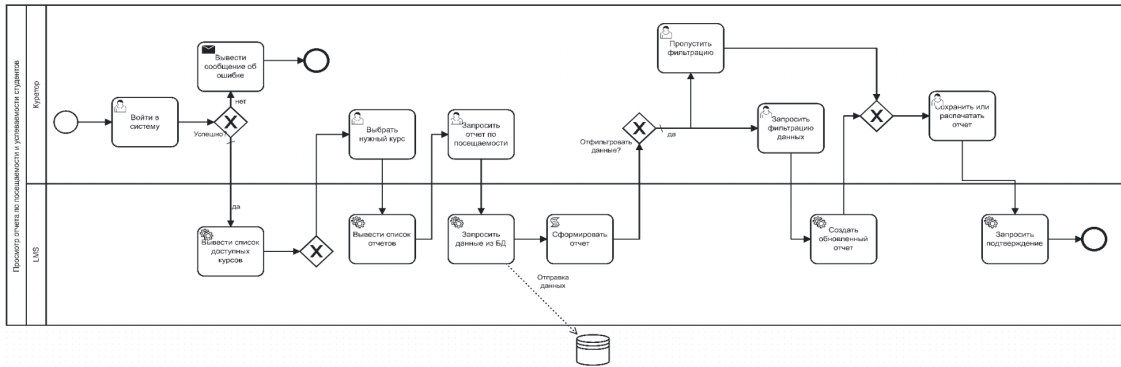
Классов (Class)

Состояний (State)

Sequence Diagram



BPMN



State

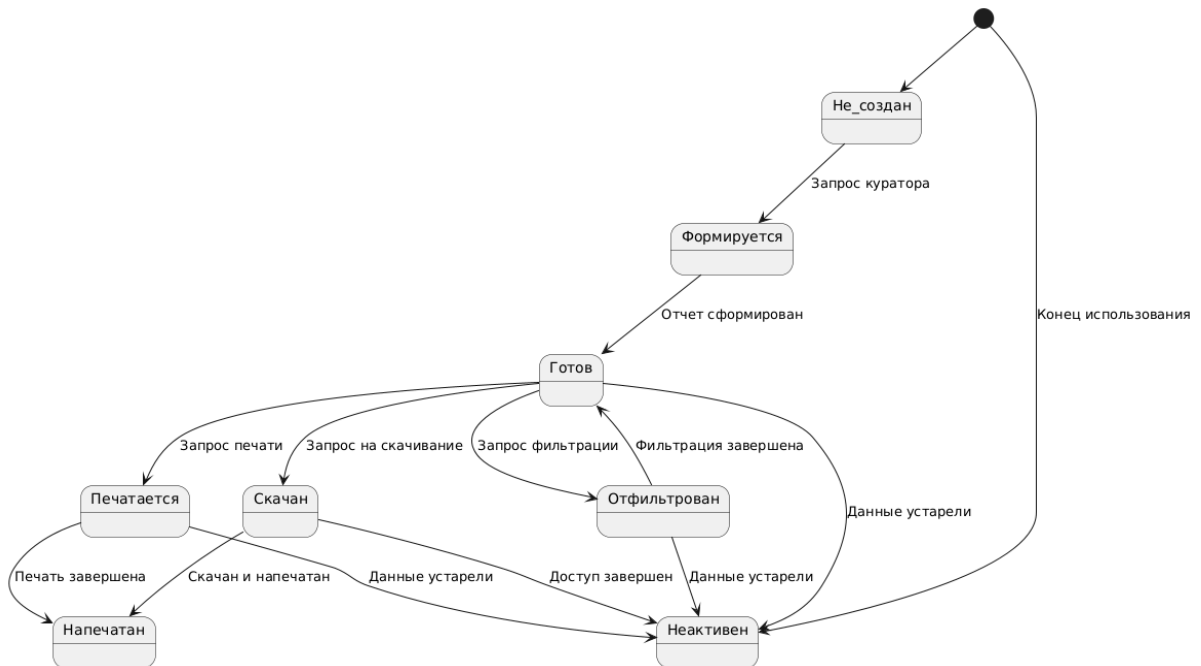
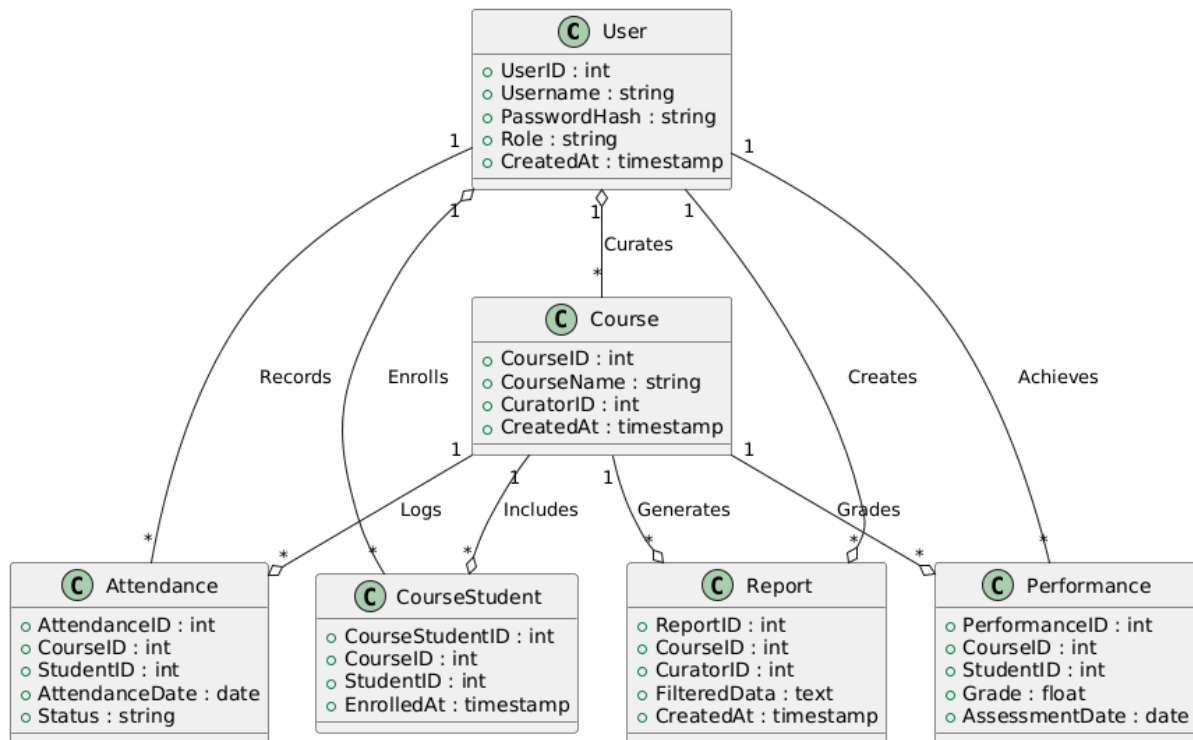
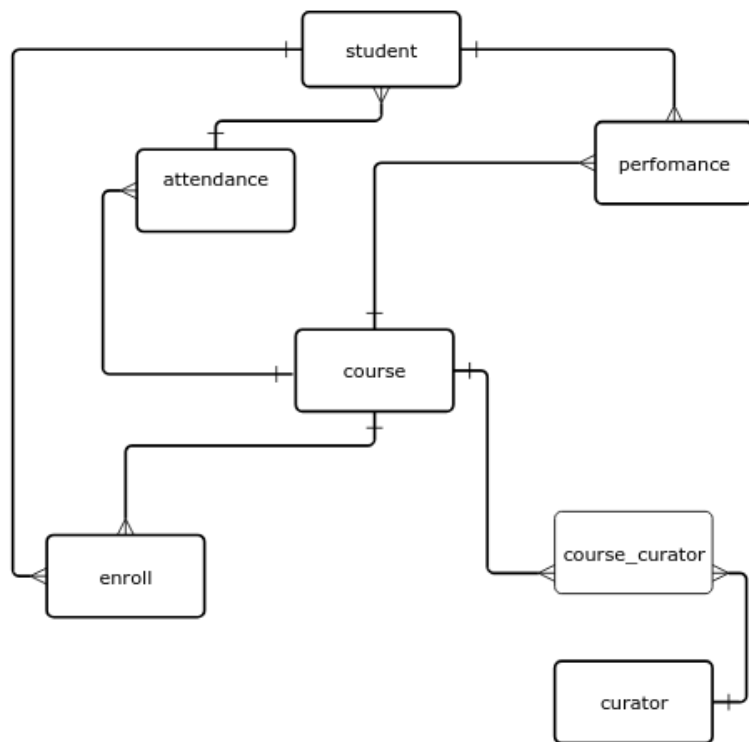


Диаграмма классов



5. ER-диаграмма:

Концептуальная модель БД



Логическая модель БД

<https://dbdiagram.io>

```

TABLE curator {
  id_curator INT pk
  login VARCHAR
  password VARCHAR
  first_name VARCHAR
  last_name VARCHAR
}
TABLE course_curator {
  id_course_curator int
  id_curator INT
  id_course INT
  date date
}
TABLE course {
  id_course INT pk
  course_name VARCHAR
  course_description TEXT
  start_date DATE
  end_date DATE
  id_curator INT
}
  
```

```

TABLE student {
  id_student INT pk
  first_name VARCHAR
  last_name VARCHAR
  email VARCHAR
}
TABLE performance {
  id_performance INT pk
  id_student INT
  id_course INT
  grade DECIMAL
  grade_date DATE
}
TABLE enroll {
  id_enroll INT pk
  id_student INT
  id_course INT
}
TABLE attendance {
  id_attendance INT pk
  id_student INT
  id_course INT
  attendance_date DATE
  status ENUM
}
Ref: "course_curator"."id_course" > "course"."id_course"
Ref: "course_curator"."id_curator" > "course"."id_curator"
Ref: "curator"."id_curator" < "course_curator"."id_curator"
Ref: "course"."id_course" < "enroll"."id_course"
Ref: "student"."id_student" < "enroll"."id_student"
Ref: "attendance"."id_student" > "student"."id_student"
Ref: "performance"."id_student" > "student"."id_student"
Ref: "course"."id_course" < "attendance"."id_course"
Ref: "performance"."id_course" > "course"."id_course"

```

